Encoding_images_and_style

April 11, 2026

[1]:
```
import torch
```

[2]:
```
torch.finfo(torch.float32).eps
```

[2]: 1.1920928955078125e-07

[3]:
```
# from google.colab import drive
# drive.mount('../../Drive/MyDrive/GAN/ENCI')

import os

drive_path = r"../../Drive/MyDrive/ENCI"
os.makedirs(drive_path, exist_ok=True)
```

[4]:
```
EPS = torch.finfo(torch.float32).eps

""" GPU """
# Enable CUDA, set tensor type and device
import torch

use_cuda = True
use_cuda = use_cuda and torch.cuda.is_available()
print("CUDA", use_cuda)
if use_cuda:
    dtype = torch.cuda.FloatTensor
    device = torch.device('cuda:0')
else:
    dtype = torch.FloatTensor
    device = torch.device('cpu')
```

```
CUDA False
```

[5]:
```
# !mkdir ../../Drive/MyDrive/ENCI\ Drive/aae

from torchvision.ops import nms
print(nms)
```

```
<function nms at 0x0000016C2EAF4B80>
```

[6]:
```
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from torch import autograd
import torch.nn as nn
from torch.utils.data import DataLoader, dataset
from torchvision.datasets import MNIST
import torchvision.transforms as T

from tqdm.notebook import trange
from torchsummary import summary
import argparse
import os
import time
```

[7]:
```
batch_size = 1024
n_classes = 10
```

[8]:
```
train_loader = torch.utils.data.DataLoader(
    MNIST(
        'Data/',
        train=True,
        download=True,
        transform=T.Compose([
                T.transforms.ToTensor(),
                T.Normalize((0.1307,), (0.3081,))  # mean and std deviation of␣
 ↪the MNIST dataset
        ])
    ),
    batch_size=batch_size,
    shuffle=True
)

val_loader = torch.utils.data.DataLoader(
    MNIST(
        'Val/',
        train=False,
        download=True,
        transform=T.Compose([
                T.transforms.ToTensor(),
                T.Normalize((0.1307,), (0.3081,))
        ])
    ),
    batch_size=batch_size,
    shuffle=False
```

```
)
```

[19]:
```
supervised = True
dims = 10  #if not supervised else 2  # size of representation layer
Model_dir = '../../Drive/MyDrive/aae/'
Fig_dir = '../../Drive/MyDrive/aae/'
```

[10]:
```
from torch import nn
import torch.nn.functional as F


class Encoder(nn.Module):
    '''Q-net'''
    def __init__(self, dim_input, hidden_dim, dim_z):
        super(Encoder, self).__init__()
        self.dim_input = dim_input  # image size
        self.hidden_dim = hidden_dim
        self.dim_z = dim_z
        self.network = []
        self.network.extend([
            nn.Linear(self.dim_input, self.hidden_dim),
            nn.Dropout(p=0.2),
            nn.ReLU(),
            nn.Linear(self.hidden_dim, self.hidden_dim),
            nn.Dropout(p=0.2),
            nn.ReLU(),
            nn.Linear(self.hidden_dim, self.dim_z),
        ])
        self.network = nn.Sequential(*self.network)
    def forward(self, x):
        z = self.network(x)
        return z

class Decoder(nn.Module):
    '''P-net'''
    def __init__(self, dim_input , hidden_dim, dim_z, supervised=False):
        super(Decoder, self).__init__()
        self.dim_input = dim_input
        self.hidden_dim = hidden_dim
        self.dim_z = dim_z
        self.supervised = supervised
        self.network = []
        #input_dim = self.dim_z if not self.supervised else self.dim_z +␣
 ↪n_classes
        input_dim = self.dim_z + n_classes   # because supervised=True
        self.network.extend([
            nn.Linear(input_dim, self.hidden_dim),
```

```
            nn.ReLU(),
            nn.Dropout(p=0.2),
            nn.Linear(self.hidden_dim, self.hidden_dim),
            nn.ReLU(),
            nn.Dropout(p=0.2),
            nn.Linear(self.hidden_dim, self.dim_input),
            nn.Sigmoid(),
        ])
        self.network = nn.Sequential(*self.network)
    def forward(self, z):
        x_recon = self.network(z)
        return x_recon

class Discriminator(nn.Module):
    def __init__(self, dims, dim_h):
        super(Discriminator,self).__init__()
        self.dim_z = dims
        self.dim_h = dim_h
        self.network = []
        self.network.extend([
            nn.Linear(self.dim_z, self.dim_h),
            nn.ReLU(),
            nn.Dropout(p=0.2),
            nn.Linear(self.dim_h, self.dim_h),
            nn.ReLU(),
            nn.Dropout(p=0.2),
            nn.Linear(self.dim_h, 1),
            nn.Sigmoid(),
        ])
        self.network = nn.Sequential(*self.network)

    def forward(self, z):
        disc = self.network(z)
        return disc
```

[11]:
```
def one_hot_encoding(labels, n_classes=10):
    cat = np.array(labels.data.tolist())
    cat = np.eye(n_classes)[cat].astype('float32')
    cat = torch.from_numpy(cat)
    return autograd.Variable(cat)
```

[12]:
```
autoencoder_loss = nn.MSELoss()

def train_validate(
        encoder,
        decoder,
        discriminator,
```

```
        dataloader,
        optim_encoder,
        optim_decoder,
        optim_D,
        train
    ):
    total_rec_loss = 0
    total_disc_loss = 0
    total_gen_loss = 0
    if train:
        encoder.train()
        decoder.train()
        discriminator.train()
    else:
        encoder.eval()
        decoder.eval()
        discriminator.eval()

    iteration = 0
    for (data, labels) in dataloader:
        # Reconstruction loss:
        for p in discriminator.parameters():
            p.requires_grad = False

        real_data = autograd.Variable(data).to(device).view(-1, 784)
        encoding = encoder(real_data)

        if decoder.supervised:
            label_hot = one_hot_encoding(labels, n_classes=10).to(device)
            restored_image = decoder(torch.cat((label_hot, encoding), 1))
        else:
            restored_image = decoder(encoding)

        # autoencoder_loss or F.binary_cross_entropy
        reconstruction_loss = autoencoder_loss(restored_image, real_data)
        total_rec_loss += reconstruction_loss.item()
        if train:
            optim_encoder.zero_grad()
            optim_decoder.zero_grad()
            reconstruction_loss.backward()
            optim_encoder.step()
            optim_decoder.step()

        # Discriminator loss:
        # i) latent representation
        encoder.eval()
        z_real_gauss = autograd.Variable(
```

```
            torch.randn(data.size()[0], dims)
        ).to(device)
        z_fake_gauss = encoder(real_data)
        # ii) feed into discriminator
        D_real_gauss = discriminator(z_real_gauss)
        D_fake_gauss = discriminator(z_fake_gauss)

        D_loss = -torch.mean(  # cross entropy
            torch.log(D_real_gauss + EPS) +
            torch.log(1 - D_fake_gauss + EPS)
        )
        total_disc_loss += D_loss.item()

        if train:
            optim_D.zero_grad()
            D_loss.backward()
            optim_D.step()

        # Generator (encoder) loss
        if train:
            encoder.train()
        else:
            encoder.eval()
        z_fake_gauss = encoder(real_data)
        D_fake_gauss = discriminator(z_fake_gauss)

        G_loss = -torch.mean(torch.log(D_fake_gauss + EPS))
        total_gen_loss += G_loss.item()

        if train:
            optim_encoder_reg.zero_grad()
            G_loss.backward()
            optim_encoder_reg.step()

        if (iteration % 100) == 0:
            print(
                'reconstruction loss: %.4f, discriminator loss: %.4f, generator␣
↪loss: %.4f' %
                (reconstruction_loss.item(), D_loss.item(), G_loss.item()))

        iteration += 1

    M = len(dataloader.dataset)
    return total_rec_loss / M, total_disc_loss / M, total_gen_loss / M
```

[13]:
```
encoder = Encoder(784, 1000, dims).to(device)
decoder = Decoder(784, 1000, dims, supervised=supervised).to(device)
discriminator = Discriminator(dims, 500).to(device)
```

[14]:
```
lr = 0.00025
```

[15]:
```
#encode/decode optimizers
optim_encoder = torch.optim.Adam(encoder.parameters(), lr=lr)
optim_decoder = torch.optim.Adam(decoder.parameters(), lr=lr)
optim_D = torch.optim.Adam(discriminator.parameters(), lr=lr)
optim_encoder_reg = torch.optim.Adam(encoder.parameters(), lr=lr * 0.1)
```

[16]:
```
n_epochs = 100
```

[17]:
```
train_loss = []
val_loss = []
for epoch in trange(n_epochs):
    l1, l2, l3 = train_validate(
        encoder, decoder, discriminator,
        train_loader, optim_encoder, optim_decoder,
        optim_D, train=True
    )
    print('epoch: {} ---- training loss: {:.8f}'.format(epoch, l1))
    train_loss.append(l1)

    if (epoch % 5) == 0:
        l1, l2, l3 = train_validate(
            encoder, decoder, discriminator,
            val_loader, optim_encoder,
            optim_decoder, optim_D, False
        )
        print('epoch: {} ---- validation loss: {:.8f}'.format(epoch, l1))
        val_loss.append(l1)
```

```
  0%|          | 0/100 [00:00<?, ?it/s]

reconstruction loss: 1.2556, discriminator loss: 1.3817, generator loss: 0.6612
epoch: 0 ---- training loss: 0.00085701
reconstruction loss: 0.7735, discriminator loss: 3.3316, generator loss: 0.0835
epoch: 0 ---- validation loss: 0.00080011
reconstruction loss: 0.8050, discriminator loss: 3.4377, generator loss: 0.1624
epoch: 1 ---- training loss: 0.00075778
reconstruction loss: 0.7335, discriminator loss: 2.5332, generator loss: 0.2229
epoch: 2 ---- training loss: 0.00068294
reconstruction loss: 0.6740, discriminator loss: 2.3565, generator loss: 0.2825
epoch: 3 ---- training loss: 0.00063813
reconstruction loss: 0.6421, discriminator loss: 2.4360, generator loss: 0.2938
epoch: 4 ---- training loss: 0.00061291
```

```
reconstruction loss: 0.6069, discriminator loss: 2.4110, generator loss: 0.3104
epoch: 5 ---- training loss: 0.00059792
reconstruction loss: 0.5700, discriminator loss: 2.3484, generator loss: 0.2748
epoch: 5 ---- validation loss: 0.00059834
reconstruction loss: 0.6016, discriminator loss: 2.4031, generator loss: 0.3259
epoch: 6 ---- training loss: 0.00058875
reconstruction loss: 0.5867, discriminator loss: 2.3858, generator loss: 0.3193
epoch: 7 ---- training loss: 0.00057930
reconstruction loss: 0.5862, discriminator loss: 2.4505, generator loss: 0.3282
epoch: 8 ---- training loss: 0.00057114
reconstruction loss: 0.5714, discriminator loss: 2.3876, generator loss: 0.3433
epoch: 9 ---- training loss: 0.00056531
reconstruction loss: 0.5748, discriminator loss: 2.4433, generator loss: 0.3207
epoch: 10 ---- training loss: 0.00056077
reconstruction loss: 0.5341, discriminator loss: 2.3893, generator loss: 0.2646
epoch: 10 ---- validation loss: 0.00056160
reconstruction loss: 0.5615, discriminator loss: 2.5270, generator loss: 0.3096
epoch: 11 ---- training loss: 0.00055597
reconstruction loss: 0.5642, discriminator loss: 2.4884, generator loss: 0.3043
epoch: 12 ---- training loss: 0.00055136
reconstruction loss: 0.5520, discriminator loss: 2.5218, generator loss: 0.3295
epoch: 13 ---- training loss: 0.00054803
reconstruction loss: 0.5548, discriminator loss: 2.5891, generator loss: 0.3357
epoch: 14 ---- training loss: 0.00054508
reconstruction loss: 0.5468, discriminator loss: 2.6344, generator loss: 0.3060
epoch: 15 ---- training loss: 0.00054276
reconstruction loss: 0.5167, discriminator loss: 2.5699, generator loss: 0.2424
epoch: 15 ---- validation loss: 0.00054482
reconstruction loss: 0.5531, discriminator loss: 2.6407, generator loss: 0.3217
epoch: 16 ---- training loss: 0.00054113
reconstruction loss: 0.5511, discriminator loss: 2.7020, generator loss: 0.3072
epoch: 17 ---- training loss: 0.00053965
reconstruction loss: 0.5485, discriminator loss: 2.6755, generator loss: 0.3011
epoch: 18 ---- training loss: 0.00053836
reconstruction loss: 0.5473, discriminator loss: 2.7699, generator loss: 0.3117
epoch: 19 ---- training loss: 0.00053724
reconstruction loss: 0.5452, discriminator loss: 2.7728, generator loss: 0.3106
epoch: 20 ---- training loss: 0.00053601
reconstruction loss: 0.5107, discriminator loss: 2.7687, generator loss: 0.2157
epoch: 20 ---- validation loss: 0.00053880
reconstruction loss: 0.5412, discriminator loss: 2.8810, generator loss: 0.2921
epoch: 21 ---- training loss: 0.00053471
reconstruction loss: 0.5418, discriminator loss: 2.9268, generator loss: 0.2626
epoch: 22 ---- training loss: 0.00053374
reconstruction loss: 0.5350, discriminator loss: 2.9734, generator loss: 0.2949
epoch: 23 ---- training loss: 0.00053285
reconstruction loss: 0.5436, discriminator loss: 3.0101, generator loss: 0.2761
epoch: 24 ---- training loss: 0.00053191
```

```
reconstruction loss: 0.5355, discriminator loss: 3.0329, generator loss: 0.2651
epoch: 25 ---- training loss: 0.00053105
reconstruction loss: 0.5064, discriminator loss: 3.0053, generator loss: 0.1901
epoch: 25 ---- validation loss: 0.00053444
reconstruction loss: 0.5364, discriminator loss: 3.1481, generator loss: 0.2708
epoch: 26 ---- training loss: 0.00053024
reconstruction loss: 0.5415, discriminator loss: 3.0875, generator loss: 0.2683
epoch: 27 ---- training loss: 0.00052932
reconstruction loss: 0.5383, discriminator loss: 3.2061, generator loss: 0.2462
epoch: 28 ---- training loss: 0.00052871
reconstruction loss: 0.5349, discriminator loss: 3.1978, generator loss: 0.2474
epoch: 29 ---- training loss: 0.00052796
reconstruction loss: 0.5330, discriminator loss: 3.3431, generator loss: 0.2400
epoch: 30 ---- training loss: 0.00052716
reconstruction loss: 0.5027, discriminator loss: 3.1833, generator loss: 0.1734
epoch: 30 ---- validation loss: 0.00053098
reconstruction loss: 0.5355, discriminator loss: 3.3198, generator loss: 0.2475
epoch: 31 ---- training loss: 0.00052645
reconstruction loss: 0.5343, discriminator loss: 3.3352, generator loss: 0.2215
epoch: 32 ---- training loss: 0.00052588
reconstruction loss: 0.5401, discriminator loss: 3.3717, generator loss: 0.2460
epoch: 33 ---- training loss: 0.00052525
reconstruction loss: 0.5329, discriminator loss: 3.4193, generator loss: 0.2263
epoch: 34 ---- training loss: 0.00052481
reconstruction loss: 0.5362, discriminator loss: 3.4142, generator loss: 0.2354
epoch: 35 ---- training loss: 0.00052422
reconstruction loss: 0.5004, discriminator loss: 3.4909, generator loss: 0.1503
epoch: 35 ---- validation loss: 0.00052866
reconstruction loss: 0.5330, discriminator loss: 3.5956, generator loss: 0.2250
epoch: 36 ---- training loss: 0.00052369
reconstruction loss: 0.5409, discriminator loss: 3.6307, generator loss: 0.2151
epoch: 37 ---- training loss: 0.00052331
reconstruction loss: 0.5332, discriminator loss: 3.7745, generator loss: 0.2274
epoch: 38 ---- training loss: 0.00052301
reconstruction loss: 0.5289, discriminator loss: 3.6887, generator loss: 0.2129
epoch: 39 ---- training loss: 0.00052260
reconstruction loss: 0.5278, discriminator loss: 3.7897, generator loss: 0.2173
epoch: 40 ---- training loss: 0.00052231
reconstruction loss: 0.4989, discriminator loss: 3.7465, generator loss: 0.1317
epoch: 40 ---- validation loss: 0.00052710
reconstruction loss: 0.5215, discriminator loss: 3.7658, generator loss: 0.2168
epoch: 41 ---- training loss: 0.00052203
reconstruction loss: 0.5336, discriminator loss: 3.9013, generator loss: 0.2127
epoch: 42 ---- training loss: 0.00052155
reconstruction loss: 0.5290, discriminator loss: 3.9160, generator loss: 0.2041
epoch: 43 ---- training loss: 0.00052138
reconstruction loss: 0.5298, discriminator loss: 4.0634, generator loss: 0.2040
epoch: 44 ---- training loss: 0.00052097
```

```
reconstruction loss: 0.5340, discriminator loss: 3.9623, generator loss: 0.2028
epoch: 45 ---- training loss: 0.00052076
reconstruction loss: 0.4976, discriminator loss: 4.0537, generator loss: 0.1175
epoch: 45 ---- validation loss: 0.00052575
reconstruction loss: 0.5260, discriminator loss: 4.1243, generator loss: 0.1890
epoch: 46 ---- training loss: 0.00052053
reconstruction loss: 0.5249, discriminator loss: 4.0590, generator loss: 0.1940
epoch: 47 ---- training loss: 0.00052026
reconstruction loss: 0.5263, discriminator loss: 4.0464, generator loss: 0.1886
epoch: 48 ---- training loss: 0.00051997
reconstruction loss: 0.5305, discriminator loss: 4.3007, generator loss: 0.1680
epoch: 49 ---- training loss: 0.00051973
reconstruction loss: 0.5336, discriminator loss: 4.2784, generator loss: 0.1701
epoch: 50 ---- training loss: 0.00051947
reconstruction loss: 0.4966, discriminator loss: 4.4140, generator loss: 0.0998
epoch: 50 ---- validation loss: 0.00052493
reconstruction loss: 0.5230, discriminator loss: 4.4607, generator loss: 0.1596
epoch: 51 ---- training loss: 0.00051911
reconstruction loss: 0.5306, discriminator loss: 4.4602, generator loss: 0.1775
epoch: 52 ---- training loss: 0.00051866
reconstruction loss: 0.5212, discriminator loss: 4.4071, generator loss: 0.1707
epoch: 53 ---- training loss: 0.00051861
reconstruction loss: 0.5240, discriminator loss: 4.6401, generator loss: 0.1602
epoch: 54 ---- training loss: 0.00051840
reconstruction loss: 0.5256, discriminator loss: 4.6492, generator loss: 0.1475
epoch: 55 ---- training loss: 0.00051811
reconstruction loss: 0.4952, discriminator loss: 4.6656, generator loss: 0.0921
epoch: 55 ---- validation loss: 0.00052358
reconstruction loss: 0.5290, discriminator loss: 4.7181, generator loss: 0.1565
epoch: 56 ---- training loss: 0.00051789
reconstruction loss: 0.5245, discriminator loss: 4.7379, generator loss: 0.1623
epoch: 57 ---- training loss: 0.00051761
reconstruction loss: 0.5249, discriminator loss: 4.6981, generator loss: 0.1591
epoch: 58 ---- training loss: 0.00051744
reconstruction loss: 0.5199, discriminator loss: 4.9776, generator loss: 0.1441
epoch: 59 ---- training loss: 0.00051719
reconstruction loss: 0.5220, discriminator loss: 5.0004, generator loss: 0.1461
epoch: 60 ---- training loss: 0.00051706
reconstruction loss: 0.4946, discriminator loss: 5.0428, generator loss: 0.0812
epoch: 60 ---- validation loss: 0.00052292
reconstruction loss: 0.5241, discriminator loss: 4.8997, generator loss: 0.1595
epoch: 61 ---- training loss: 0.00051687
reconstruction loss: 0.5253, discriminator loss: 4.8648, generator loss: 0.1426
epoch: 62 ---- training loss: 0.00051682
reconstruction loss: 0.5296, discriminator loss: 5.1858, generator loss: 0.1396
epoch: 63 ---- training loss: 0.00051657
reconstruction loss: 0.5166, discriminator loss: 5.1332, generator loss: 0.1342
epoch: 64 ---- training loss: 0.00051638
```

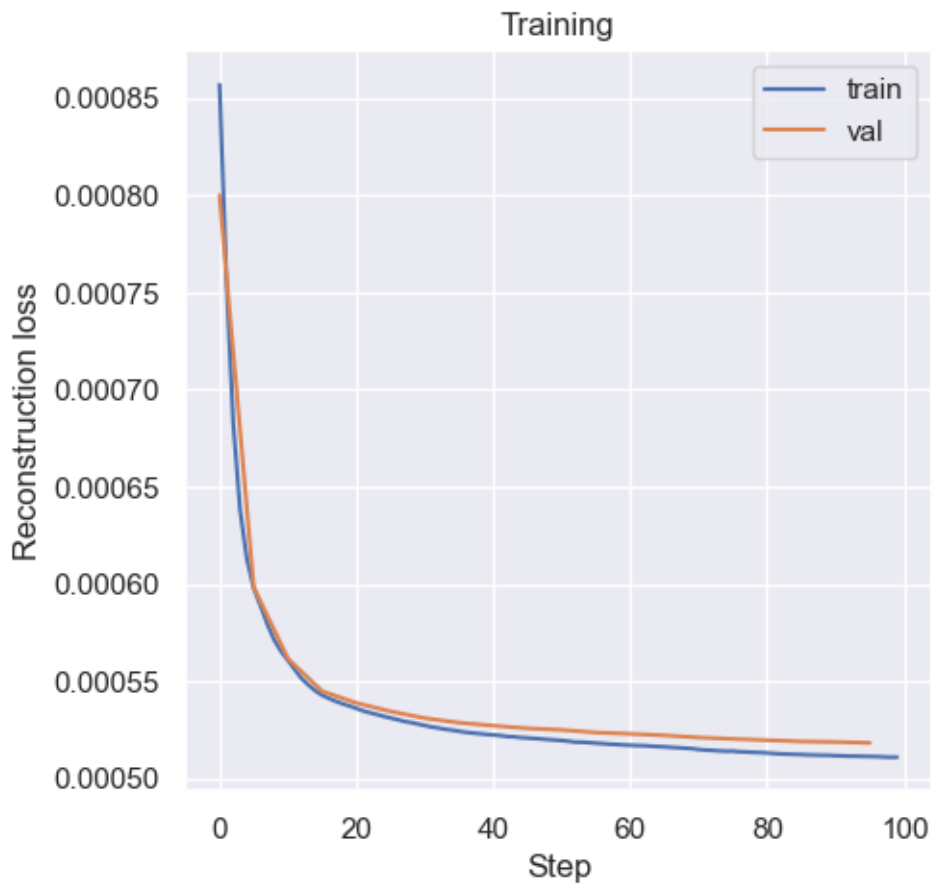
```
reconstruction loss: 0.5273, discriminator loss: 5.1488, generator loss: 0.1129
epoch: 65 ---- training loss: 0.00051623
reconstruction loss: 0.4937, discriminator loss: 5.3724, generator loss: 0.0737
epoch: 65 ---- validation loss: 0.00052210
reconstruction loss: 0.5249, discriminator loss: 5.4529, generator loss: 0.1091
epoch: 66 ---- training loss: 0.00051596
reconstruction loss: 0.5197, discriminator loss: 5.6162, generator loss: 0.1096
epoch: 67 ---- training loss: 0.00051576
reconstruction loss: 0.5228, discriminator loss: 5.5029, generator loss: 0.1155
epoch: 68 ---- training loss: 0.00051550
reconstruction loss: 0.5242, discriminator loss: 5.7870, generator loss: 0.1049
epoch: 69 ---- training loss: 0.00051532
reconstruction loss: 0.5261, discriminator loss: 5.4939, generator loss: 0.1242
epoch: 70 ---- training loss: 0.00051487
reconstruction loss: 0.4924, discriminator loss: 5.6761, generator loss: 0.0680
epoch: 70 ---- validation loss: 0.00052103
reconstruction loss: 0.5306, discriminator loss: 5.6428, generator loss: 0.1309
epoch: 71 ---- training loss: 0.00051456
reconstruction loss: 0.5184, discriminator loss: 5.5520, generator loss: 0.1259
epoch: 72 ---- training loss: 0.00051442
reconstruction loss: 0.5228, discriminator loss: 5.7497, generator loss: 0.1257
epoch: 73 ---- training loss: 0.00051416
reconstruction loss: 0.5263, discriminator loss: 5.9797, generator loss: 0.0994
epoch: 74 ---- training loss: 0.00051399
reconstruction loss: 0.5106, discriminator loss: 5.9592, generator loss: 0.1130
epoch: 75 ---- training loss: 0.00051402
reconstruction loss: 0.4915, discriminator loss: 6.1192, generator loss: 0.0599
epoch: 75 ---- validation loss: 0.00052028
reconstruction loss: 0.5173, discriminator loss: 6.1078, generator loss: 0.1127
epoch: 76 ---- training loss: 0.00051369
reconstruction loss: 0.5262, discriminator loss: 5.9502, generator loss: 0.1109
epoch: 77 ---- training loss: 0.00051359
reconstruction loss: 0.5206, discriminator loss: 6.2895, generator loss: 0.1073
epoch: 78 ---- training loss: 0.00051338
reconstruction loss: 0.5224, discriminator loss: 6.3943, generator loss: 0.1029
epoch: 79 ---- training loss: 0.00051328
reconstruction loss: 0.5197, discriminator loss: 6.2566, generator loss: 0.0911
epoch: 80 ---- training loss: 0.00051307
reconstruction loss: 0.4908, discriminator loss: 6.4508, generator loss: 0.0565
epoch: 80 ---- validation loss: 0.00051965
reconstruction loss: 0.5234, discriminator loss: 6.4340, generator loss: 0.1121
epoch: 81 ---- training loss: 0.00051279
reconstruction loss: 0.5285, discriminator loss: 6.4674, generator loss: 0.0950
epoch: 82 ---- training loss: 0.00051258
reconstruction loss: 0.5217, discriminator loss: 6.5159, generator loss: 0.0975
epoch: 83 ---- training loss: 0.00051246
reconstruction loss: 0.5211, discriminator loss: 6.7397, generator loss: 0.0879
epoch: 84 ---- training loss: 0.00051234
```

```
reconstruction loss: 0.5229, discriminator loss: 6.6736, generator loss: 0.0945
epoch: 85 ---- training loss: 0.00051226
reconstruction loss: 0.4902, discriminator loss: 6.8230, generator loss: 0.0505
epoch: 85 ---- validation loss: 0.00051899
reconstruction loss: 0.5135, discriminator loss: 6.7222, generator loss: 0.0895
epoch: 86 ---- training loss: 0.00051206
reconstruction loss: 0.5183, discriminator loss: 7.0396, generator loss: 0.0959
epoch: 87 ---- training loss: 0.00051202
reconstruction loss: 0.5216, discriminator loss: 6.8370, generator loss: 0.0823
epoch: 88 ---- training loss: 0.00051190
reconstruction loss: 0.5199, discriminator loss: 6.9762, generator loss: 0.0851
epoch: 89 ---- training loss: 0.00051186
reconstruction loss: 0.5219, discriminator loss: 7.1842, generator loss: 0.0840
epoch: 90 ---- training loss: 0.00051168
reconstruction loss: 0.4900, discriminator loss: 7.2579, generator loss: 0.0456
epoch: 90 ---- validation loss: 0.00051874
reconstruction loss: 0.5300, discriminator loss: 7.1053, generator loss: 0.0937
epoch: 91 ---- training loss: 0.00051160
reconstruction loss: 0.5255, discriminator loss: 7.1908, generator loss: 0.0820
epoch: 92 ---- training loss: 0.00051140
reconstruction loss: 0.5189, discriminator loss: 7.1879, generator loss: 0.0776
epoch: 93 ---- training loss: 0.00051144
reconstruction loss: 0.5189, discriminator loss: 7.3959, generator loss: 0.0839
epoch: 94 ---- training loss: 0.00051131
reconstruction loss: 0.5215, discriminator loss: 7.4047, generator loss: 0.0749
epoch: 95 ---- training loss: 0.00051127
reconstruction loss: 0.4895, discriminator loss: 7.6487, generator loss: 0.0415
epoch: 95 ---- validation loss: 0.00051825
reconstruction loss: 0.5174, discriminator loss: 7.4562, generator loss: 0.0647
epoch: 96 ---- training loss: 0.00051118
reconstruction loss: 0.5204, discriminator loss: 7.6009, generator loss: 0.0829
epoch: 97 ---- training loss: 0.00051103
reconstruction loss: 0.5138, discriminator loss: 7.5522, generator loss: 0.0773
epoch: 98 ---- training loss: 0.00051088
reconstruction loss: 0.5252, discriminator loss: 7.6291, generator loss: 0.0677
epoch: 99 ---- training loss: 0.00051089
```

[21]:

```
%matplotlib inline
from matplotlib import pyplot as plt
import seaborn as sns

sns.set()
plt.rcParams['figure.figsize'] = 5, 5
plt.plot(np.arange(len(train_loss)), train_loss, label='train')
plt.plot(np.arange(0, len(val_loss) * 5, 5), val_loss, label='val')
plt.title('Training')
plt.xlabel('Step')
```

```
plt.ylabel('Reconstruction loss')
plt.legend()
plt.grid(True)
plt.savefig(Fig_dir + 'training_' + str(n_epochs) + 'epochs.png')
```



[22]:
```
from sklearn.manifold import TSNE


def plot_vis(X_enc, y, dims):
    '''Performing t-SNE dimensionality reduction
    and plot in 2 or 3 dimensions.
    '''
    X_tsne = TSNE(n_components=dims).fit_transform(X_enc)
    cmap = plt.get_cmap('rainbow', 10)

    if dims == 2:
        print("IN FOR 2D")
        plt.scatter(
```

```
            X_tsne[:, 0], X_tsne[:, 1],
            c=y, edgecolor='black', cmap=cmap
        )
    else:
        print("IN ELSE FOR 3D")
        fig = plt.figure()
        ax = plt.axes(projection='3d')
        p = ax.scatter3D(
            X_tsne[:, 0], X_tsne[:, 1], X_tsne[:, 2],
            c=y, cmap=cmap, edgecolor='black'
        )
        fig.colorbar(p, drawedges=True)
```

[23]:
```
# visualization of the unsupervised case
# Does our encoder distinguish the classes?
import math
n_points = 1000

X = []
y = []
for (data, labels) in val_loader:
  images = autograd.Variable(data).to(device).view(-1, 784)
  X.append(encoder(images).cpu().data.numpy())
  y.append(labels.cpu().data.numpy())
  if len(X) * images.shape[0] > n_points:
      break

X = np.concatenate(X, 0)
y = np.concatenate(y, 0)
X = X[:n_points, :]
y = y[:n_points]

print("XSHAPE: ",X.shape)
```

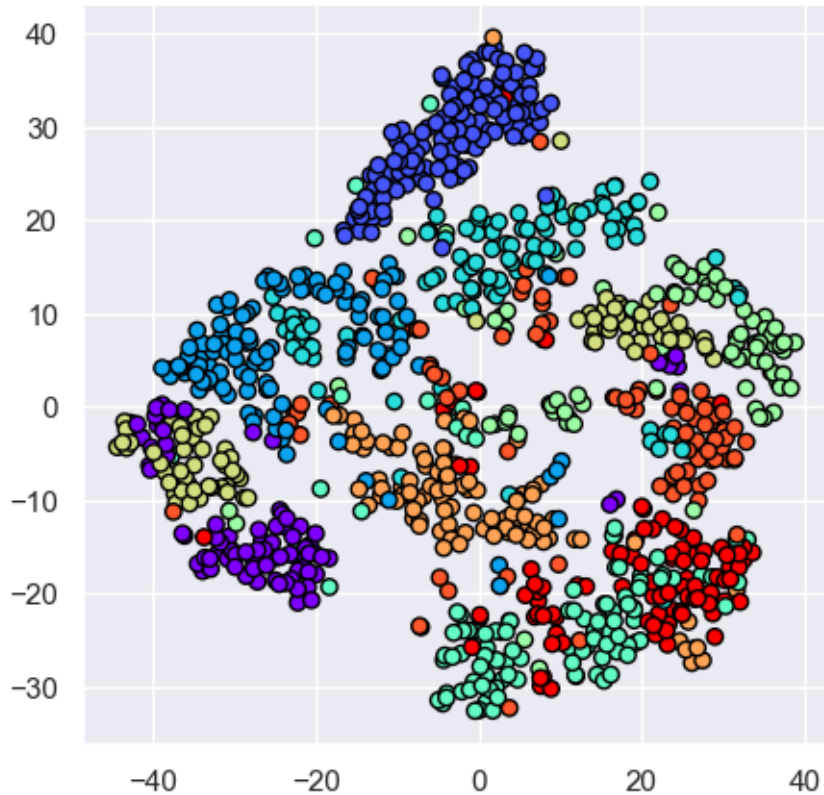
```
XSHAPE:  (1000, 10)
```

[24]:
```
print("BEGIN")
X_tsne = TSNE(n_components=3).fit_transform(X)
print("COMPLETE")
```

```
BEGIN
COMPLETE
```

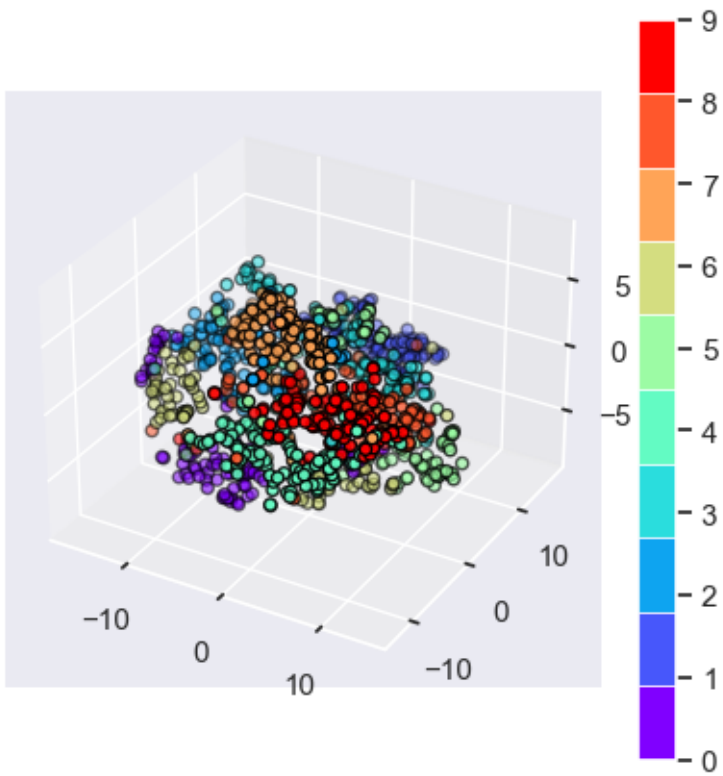
[25]:
```
plot_vis(X, y, dims=2)
```

```
IN FOR 2D
```



[26]:
```
plot_vis(X, y, dims=3)
```

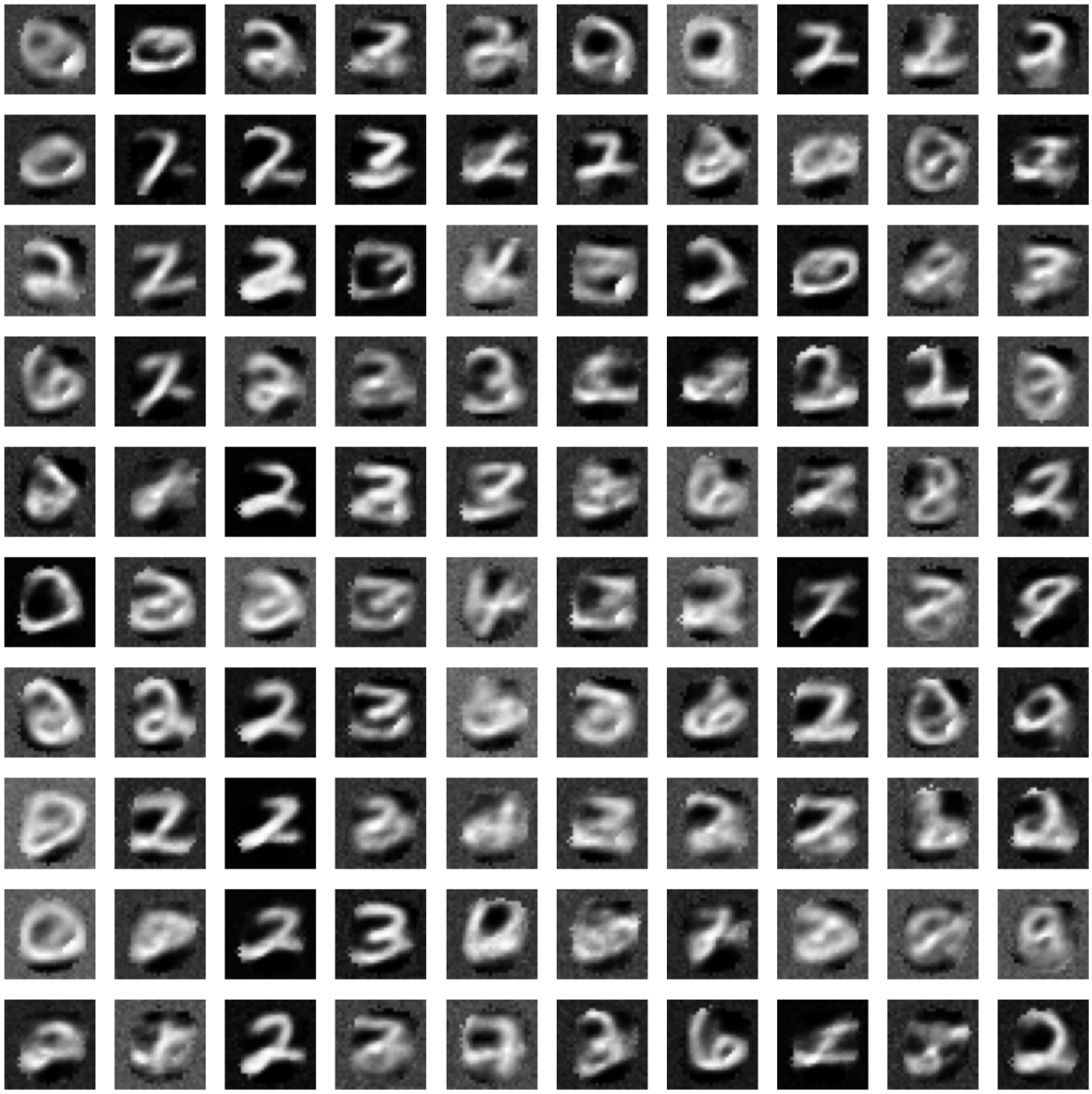
```
IN ELSE FOR 3D
```



[28]:
```
# visualization of the supervised case
# Does our decoder produce the classes and different styles?

# styles = styles + torch.randn(10, dims).reshape(1, -1).repeat(10, 1).
 ↪reshape(100, dims)
# numbers = torch.Tensor(np.eye(10)).repeat(10, 1)
# coded = torch.cat([numbers, styles], 1).to(device)
# decoded = decoder(coded).cpu().data.numpy()

# Create 10 style vectors, each 10-dimensional
styles = torch.cat([
    torch.cat([torch.linspace(-3., 3., 5).reshape(5,1),
               torch.zeros(5,9)], dim=1),
    torch.cat([torch.zeros(5,9),
               torch.linspace(-3., 3., 5).reshape(5,1)], dim=1)
], dim=0)
```

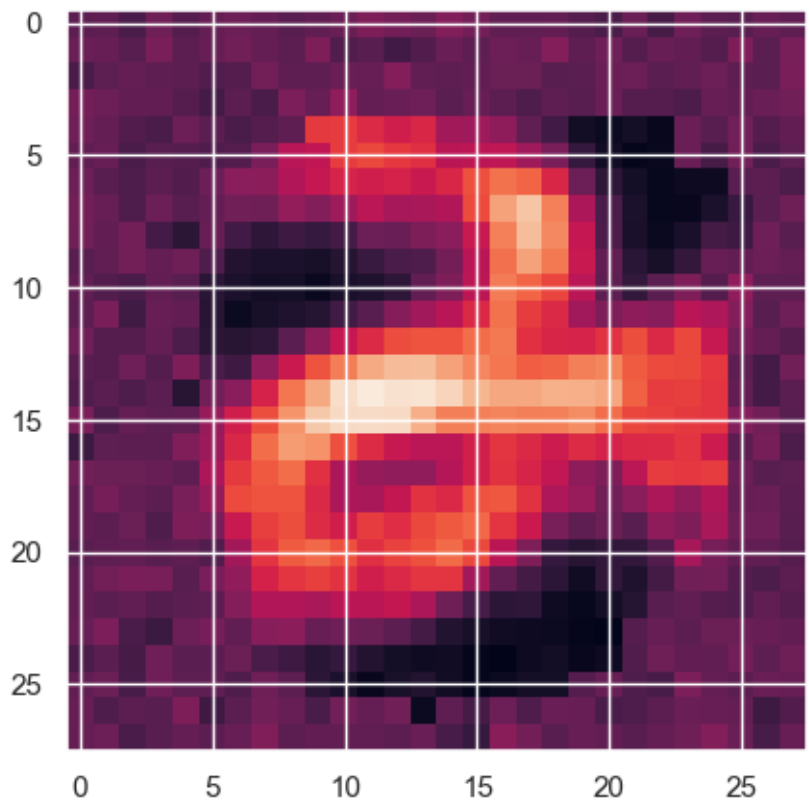
[29]:
```
# visualization of the supervised case
# Does our decoder produce the classes and different styles?

import numpy as np
```

```python
import torch
import matplotlib.pyplot as plt

# ----------------------------
# 1. Generate latent style vectors
# ----------------------------
# 100 random style vectors (10 classes × 10 samples each)
styles = torch.randn(100, dims)

# ----------------------------
# 2. Generate one-hot class labels
# ----------------------------
# 10 classes, each repeated 10 times → shape (100, 10)
numbers = torch.Tensor(np.eye(10)).repeat(10, 1)

# ----------------------------
# 3. Concatenate labels + styles
# ----------------------------
coded = torch.cat([numbers, styles], dim=1).to(device)

# ----------------------------
# 4. Decode into images
# ----------------------------
decoded = decoder(coded).cpu().data.numpy()

# ----------------------------
# 5. Plot the 100 decoded images in a 10×10 grid
# ----------------------------
fig, axes = plt.subplots(10, 10, figsize=(10, 10))

for i in range(100):
    ax = axes[i // 10, i % 10]
    ax.imshow(decoded[i].reshape(28, 28), cmap='gray')
    ax.axis('off')

plt.tight_layout()
plt.show()
```



[30]:
```
plt.imshow(decoded.reshape(10, 10, 28, 28)[0, 4])
```

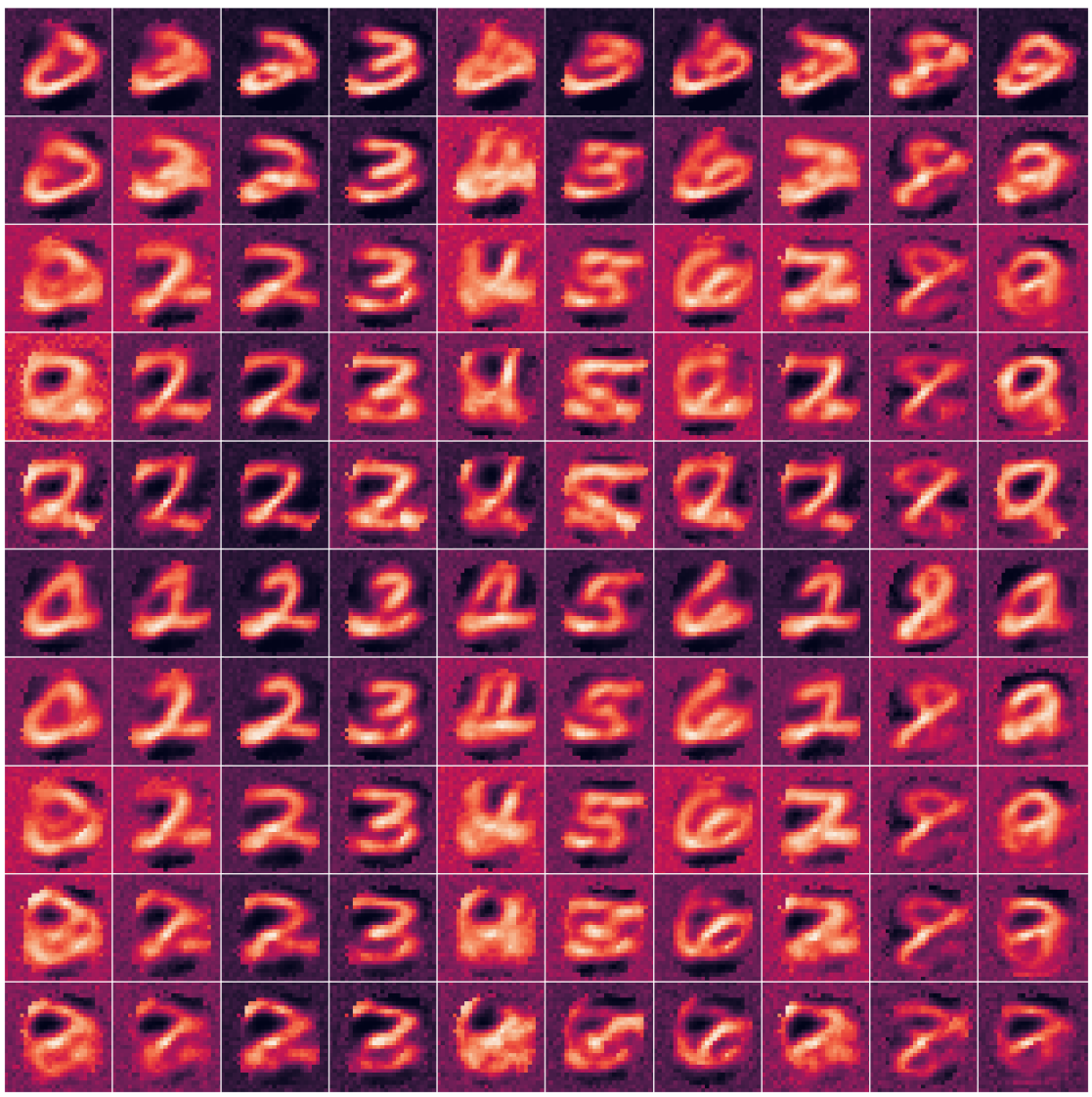
[30]: <matplotlib.image.AxesImage at 0x16c803af010>



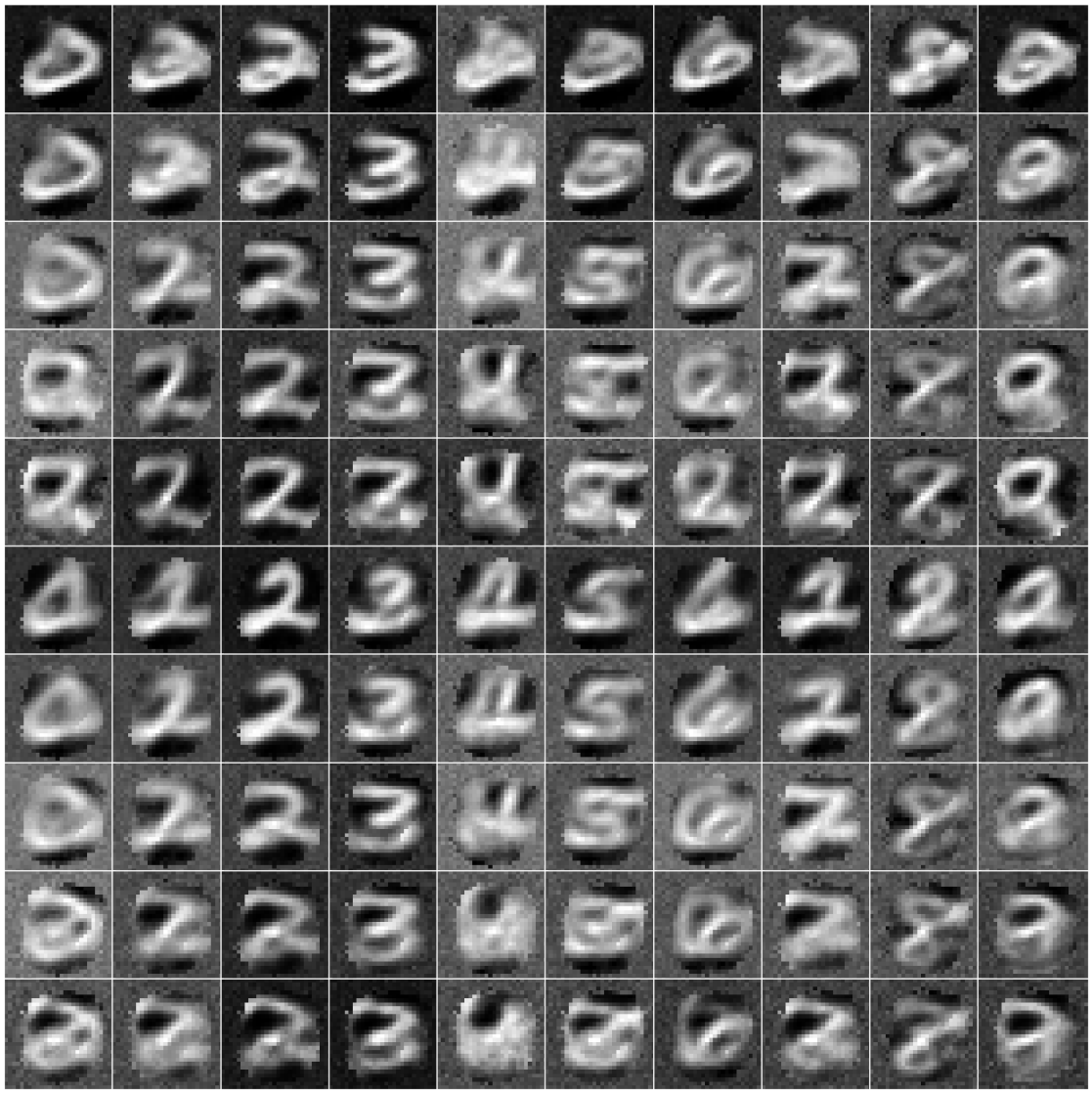
[31]:
```
# styles = torch.Tensor(
#     np.concatenate([
#         np.concatenate([np.linspace(-3., 3., 5).reshape(5, 1), np.
 ↪zeros(shape=(5, 1))], axis=1),
#         np.concatenate([np.zeros(shape=(5, 1)), np.linspace(-3., 3., 5).
 ↪reshape(5, 1)], axis=1),
#     ], axis=0)
# )

# Create 10 style vectors, each 10-dimensional
styles = torch.cat([
    torch.cat([torch.linspace(-3., 3., 5).reshape(5,1),
               torch.zeros(5,9)], dim=1),
    torch.cat([torch.zeros(5,9),
               torch.linspace(-3., 3., 5).reshape(5,1)], dim=1)
], dim=0)
```

[32]:
```
import random

nrows = styles.shape[0]
ncols = 10
```

```
fig, axes = plt.subplots(nrows, ncols, figsize=(15, 15 * nrows / 10))
numbers = torch.Tensor(np.eye(10))
for row, style in enumerate(styles):
    for col, number in enumerate(numbers):
        coded = torch.cat([number, style], -1).to(device)
        decoded = decoder(coded).cpu().data.numpy()
        ax = axes[row, col]
        ax.imshow(decoded.reshape(28, 28))
        ax.grid(False)
        ax.get_xaxis().set_visible(False)
        ax.get_yaxis().set_visible(False)

fig.tight_layout(pad=-.2)
```



[33]:
```
import random

nrows = styles.shape[0]   # number of style vectors
ncols = 10                # 10 classes

fig, axes = plt.subplots(nrows, ncols, figsize=(15, 15 * nrows / 10))

numbers = torch.Tensor(np.eye(10))   # one-hot labels

for row, style in enumerate(styles):
    for col, number in enumerate(numbers):

        # concatenate class label + style vector
        coded = torch.cat([number, style], -1).unsqueeze(0).to(device)

        # decode
        decoded = decoder(coded).cpu().data.numpy()[0]

        # plot
        ax = axes[row, col]
        ax.imshow(decoded.reshape(28, 28), cmap='gray')
        ax.grid(False)
        ax.get_xaxis().set_visible(False)
        ax.get_yaxis().set_visible(False)

fig.tight_layout(pad=-.2)
plt.show()
```



[301]: